

```

#!/bin/bash
# =====
# CODER + AUDITOR PIPELINE v2
# Loop: PLAN → APPROVE → BUILD → TEST → FIX → AUDIT → MEMORY
# Coder: qwen2.5-coder:32b (Mac)
# Auditor: qwen2.5-coder:32b (Mac) - switched from NUC (not enough RAM)
# =====

MAC="http://192.168.68.111:11434"
PROJECT_DIR=~/.AI_SYSTEM/projects/govcon_agent
SRC_DIR=$PROJECT_DIR/src
MEMORY=$PROJECT_DIR/memory/coder_memory.md
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
REPORT=$PROJECT_DIR/reports/build_${TIMESTAMP}.md
CORE_RULES=$(cat $PROJECT_DIR/TRAINING_MATERIALS/FOUNDATION/FOUNDATION_01_CORE_RULES.md)

# Make sure src dir exists
mkdir -p $SRC_DIR
mkdir -p $PROJECT_DIR/reports
mkdir -p $PROJECT_DIR/memory
mkdir -p $PROJECT_DIR/logs

# =====
# HELPER: TEST A FILE
# Returns 0 = pass, 1 = fail
# =====
test_file() {
    local filepath=$1
    local ext="${filepath##*.}"
    local result=""

    echo "  Testing: $filepath"

    # Check file exists
    if [ ! -f "$filepath" ]; then
        echo "  FAIL: File does not exist"
        return 1
    fi

    # Check file is not empty
    if [ ! -s "$filepath" ]; then
        echo "  FAIL: File is empty"
        return 1
    fi
}

```

```

# Check for placeholder content
if grep -qi "TODO\\|placeholder\\|\\.\\.\\.\\|pass$\\|your code here" "$filepath" 2>/d
    echo "    FAIL: Contains placeholder content"
    return 1
fi

# Type-specific tests
case "$ext" in
    py)
        python3 -m py_compile "$filepath" 2>/tmp/test_error.txt
        if [ $? -ne 0 ]; then
            echo "    FAIL: Python syntax error: $(cat /tmp/test_error.txt)"
            return 1
        fi
        echo "    PASS: Python syntax OK"
        ;;
    sh)
        bash -n "$filepath" 2>/tmp/test_error.txt
        if [ $? -ne 0 ]; then
            echo "    FAIL: Shell syntax error: $(cat /tmp/test_error.txt)"
            return 1
        fi
        echo "    PASS: Shell syntax OK"
        ;;
    json)
        python3 -c "import json,sys; json.load(open('$filepath'))" 2>/tmp/test_erro
        if [ $? -ne 0 ]; then
            echo "    FAIL: Invalid JSON: $(cat /tmp/test_error.txt)"
            return 1
        fi
        echo "    PASS: JSON valid"
        ;;
    md)
        echo "    PASS: Markdown exists and has content"
        ;;
    *)
        echo "    PASS: File exists and has content"
        ;;
esac

return 0
}

# -----
# HELPER: EXTRACT AND WRITE FILES FROM AI OUTPUT
# -----

```

```

write_files_from_output() {
    local output="$1"
    local written_files=()

    # Parse FILE: blocks from AI output
    while IFS= read -r line; do
        if [[ "$line" =~ ^FILE:\ (.+)\$ ]]; then
            current_file="${BASH_REMATCH[1]}"
            current_file=$(echo "$current_file" | xargs) # trim whitespace
            in_code_block=false
            file_content=""
        elif [[ "$line" == '`'* ]] && [ -n "$current_file" ]; then
            if [ "$in_code_block" = false ]; then
                in_code_block=true
            else
                # End of code block - write the file
                if [ -n "$current_file" ] && [ -n "$file_content" ]; then
                    mkdir -p "$(dirname "$current_file")"
                    echo "$file_content" > "$current_file"
                    echo "  Written: $current_file"
                    written_files+=("$current_file")
                fi
                in_code_block=false
                current_file=""
                file_content=""
            fi
        elif [ "$in_code_block" = true ]; then
            file_content+="$line$'\n'
        fi
    done <<< "$output"

    # Return written files
    printf '%s\n' "${written_files[@]}"
}

# -----
# START
# -----

clear
echo ""
echo "=====
echo "  CODER + AUDITOR PIPELINE v2"
echo "  PLAN → BUILD → TEST → FIX → AUDIT"
echo "=====
echo ""
echo "What do you want to build?"
echo ""

```

```

read -p "TASK: " TASK

if [ -z "$TASK" ]; then
    echo "No task entered. Exiting."
    exit 0
fi

# Init report
echo "# BUILD REPORT - $TIMESTAMP" > $REPORT
echo "TASK: $TASK" >> $REPORT
echo "" >> $REPORT

# -----
# STEP 1 - PLAN
# -----

echo ""
echo "[ 1/6 ] Planning..."
echo ""

PLAN=$(OLLAMA_HOST=$MAC ollama run qwen2.5-coder:32b \
"$CORE_RULES

You are a production CLI Coder agent.
TASK: $TASK
OUTPUT DIRECTORY: $SRC_DIR

STRICT RULES:
- Every file MUST have complete working code
- No empty files, no placeholders, no TODOs
- No asking questions after approval
- All file paths must be under: $SRC_DIR

Produce a build plan:
PLAN_SUMMARY: (what you will build, 2 sentences max)
APPROACH_A: (recommended - describe structure and files)
APPROACH_B: (alternative approach)
APPROACH_C: (simpler/faster alternative)
RECOMMENDATION: (which and why)
FILES_TO_CREATE: (list every file with full path under $SRC_DIR)
ESTIMATED_TIME: (minutes)")

echo "$PLAN"
echo "" >> $REPORT
echo "## PLAN" >> $REPORT
echo "$PLAN" >> $REPORT

# -----

```

```

# STEP 2 - APPROVAL GATE
# -----
echo ""
echo "=====
echo "  APPROVE PLAN?"
echo "  A = Approach A"
echo "  B = Approach B"
echo "  C = Approach C"
echo "  X = Cancel"
echo "=====
read -p "YOUR CHOICE: " CHOICE

if [[ "$CHOICE" == "X" || "$CHOICE" == "x" ]]; then
    echo "Cancelled. Nothing built."
    exit 0
fi

APPROVED_APPROACH="Approach $CHOICE"
echo ""
echo "Approved: $APPROVED_APPROACH"
echo "Building..."
echo ""

# -----
# STEP 3 - BUILD
# -----
echo "[ 2/6 ] Building..."
echo ""

BUILD_OUTPUT=$(OLLAMA_HOST=$MAC ollama run qwen2.5-coder:32b \
"$SCORE_RULES

You are a production CLI Coder agent.
TASK: $TASK
APPROVED: $APPROVED_APPROACH
PLAN: $PLAN
OUTPUT DIRECTORY: $SRC_DIR

STRICT RULES:
- Write COMPLETE working code for every file
- No placeholders, no TODOs, no empty functions
- No asking questions
- Use exact file paths under $SRC_DIR

Output EVERY file in this EXACT format:
FILE: $SRC_DIR/filename.ext
\\`

```

```
(complete working file content)
```

```
\\`\\`\\`
```

```
Write every file completely. Do not stop until all files are done.")
```

```
echo "$BUILD_OUTPUT" | head -50
```

```
echo "..."
```

```
echo "(full output saved to report)"
```

```
echo "" >> $REPORT
```

```
echo "## BUILD OUTPUT" >> $REPORT
```

```
echo "$BUILD_OUTPUT" >> $REPORT
```

```
# _____
```

```
# STEP 4 - WRITE FILES TO DISK
```

```
# _____
```

```
echo ""
```

```
echo "[ 3/6 ] Writing files to disk..."
```

```
echo ""
```

```
WRITTEN_FILES=$(write_files_from_output "$BUILD_OUTPUT")
```

```
if [ -z "$WRITTEN_FILES" ]; then
```

```
    echo "    WARNING: No files were parsed from build output"
```

```
    echo "    Asking agent to write files directly..."
```

```
    # Ask agent to write files using a simpler format
```

```
    SIMPLE_BUILD=$(OLLAMA_HOST=$MAC ollama run qwen2.5-coder:32b \
```

```
    "Write the files for this task to disk.
```

```
    TASK: $TASK
```

```
    OUTPUT DIR: $SRC_DIR
```

```
    For each file, output ONLY:
```

```
    FILE: full/path/to/file.ext
```

```
    \\`\\`\\`
```

```
    complete content
```

```
    \\`\\`\\`
```

```
    Nothing else. Start writing now.")
```

```
    WRITTEN_FILES=$(write_files_from_output "$SIMPLE_BUILD")
```

```
fi
```

```
echo ""
```

```
echo "Files written:"
```

```
echo "$WRITTEN_FILES"
```

```

# -----
# STEP 5 - TEST + AUTO-FIX LOOP (max 3 tries)
# -----

echo ""
echo "[ 4/6 ] Testing all files..."
echo ""

FAILED_FILES=()
PASSED_FILES=()
ALL_FILES_TO_TEST=()

# Collect all files to test
while IFS= read -r f; do
    [ -n "$f" ] && ALL_FILES_TO_TEST+=("$f")
done <<< "$WRITTEN_FILES"

# Also scan src dir for any new files
while IFS= read -r f; do
    ALL_FILES_TO_TEST+=("$f")
done < <(find $SRC_DIR -type f -newer $REPORT 2>/dev/null)

# Remove duplicates
ALL_FILES_TO_TEST=($(printf '%s\n' "${ALL_FILES_TO_TEST[@]}" | sort -u))

for filepath in "${ALL_FILES_TO_TEST[@]}; do
    [ -z "$filepath" ] && continue

    attempt=1
    max_attempts=3
    file_passed=false

    while [ $attempt -le $max_attempts ]; do
        if test_file "$filepath"; then
            PASSED_FILES+=("$filepath")
            file_passed=true
            break
        else
            if [ $attempt -lt $max_attempts ]; then
                echo "  Attempt $attempt failed. Auto-fixing..."

                ERROR_MSG=$(cat /tmp/test_error.txt 2>/dev/null || echo "test failed")
                FILE_CONTENT=$(cat "$filepath" 2>/dev/null || echo "file not found")

                FIX=$(OLLAMA_HOST=$MAC ollama run qwen2.5-coder:32b \
                    "Fix this file. Return ONLY the complete fixed file content, nothing else"

                FILE: $filepath
            fi
        fi
    done
done

```

```

        ERROR: $ERROR_MSG
        CURRENT CONTENT:
        $FILE_CONTENT

        Return the complete fixed file content now:")

        echo "$FIX" > "$filepath"
        echo "    Fixed. Retesting..."
    fi
    ((attempt++))
fi
done

if [ "$file_passed" = false ]; then
    FAILED_FILES+=("$filepath")
    echo "    GAVE UP after $max_attempts attempts: $filepath"
fi
done

# Test summary
echo ""
echo "===== "
echo "  TEST RESULTS"
echo "  PASSED: ${#PASSED_FILES[@]}"
echo "  FAILED: ${#FAILED_FILES[@]}"
if [ ${#FAILED_FILES[@]} -gt 0 ]; then
    echo "    NEEDS HUMAN:"
    for f in "${FAILED_FILES[@]}"; do
        echo "      - $f"
    done
fi
echo "===== "

echo "" >> $REPORT
echo "## TEST RESULTS" >> $REPORT
echo "PASSED: ${#PASSED_FILES[@]}" >> $REPORT
echo "FAILED: ${#FAILED_FILES[@]}" >> $REPORT
for f in "${FAILED_FILES[@]}"; do
    echo "NEEDS_HUMAN: $f" >> $REPORT
done

# -----
# STEP 6 - AUDIT
# -----

echo ""
echo "[ 5/6 ] Auditing..."
echo ""

```

```

# Build file list for audit
FILE_LIST=""
for f in "${PASSED_FILES[@]}"; do
    FILE_LIST+="$f: $(cat $f | head -20)\n---\n"
done

AUDIT=$(OLLAMA_HOST=$MAC ollama run qwen2.5-coder:32b \
"$SCORE_RULES

You are a production Auditor agent.
TASK: $TASK

FILES THAT PASSED TESTING:
$FILE_LIST

FAILED FILES (already flagged for human):
$(printf '%s\n' "${FAILED_FILES[@]}")

Audit the passing files for:
1. Logic errors or bugs not caught by syntax check
2. Security issues
3. Missing error handling
4. Incomplete functionality
5. Code quality issues

For each issue:
ISSUE: (description)
FILE: (which file)
SEVERITY: HIGH / MEDIUM / LOW
FIX: (exact fix or NEEDS_HUMAN)

Summary:
AUDIT_RESULT: PASS or PASS_WITH_WARNINGS or FAIL
QUALITY_SCORE: (1-10)
OVERALL: (one sentence)")

echo "$AUDIT"

echo "" >> $REPORT
echo "## AUDIT" >> $REPORT
echo "$AUDIT" >> $REPORT

# _____
# STEP 7 — MEMORY
# _____
echo ""

```

```
echo "[ 6/6 ] Logging to memory..."
```

```
cat >> $MEMORY << EOF
```

```
---
```

```
DATE: $(date)
```

```
TASK: $TASK
```

```
APPROACH: $APPROVED_APPROACH
```

```
FILES_PASSED: ${#PASSED_FILES[@]}
```

```
FILES_FAILED: ${#FAILED_FILES[@]}
```

```
NEEDS_HUMAN: $(printf '%s ' "${FAILED_FILES[@]}")
```

```
AUDIT: $(echo "$AUDIT" | grep "AUDIT_RESULT:")
```

```
QUALITY: $(echo "$AUDIT" | grep "QUALITY_SCORE:")
```

```
REPORT: $REPORT
```

```
EOF
```

```
# _____
```

```
# DONE
```

```
# _____
```

```
echo ""
```

```
echo "=====
```

```
echo "  PIPELINE COMPLETE"
```

```
echo ""
```

```
echo "   PASSED: ${#PASSED_FILES[@]} files"
```

```
if [ ${#FAILED_FILES[@]} -gt 0 ]; then
```

```
echo "   NEEDS HUMAN: ${#FAILED_FILES[@]} files"
```

```
for f in "${FAILED_FILES[@]"; do
```

```
echo "      → $f"
```

```
done
```

```
fi
```

```
echo ""
```

```
echo "  Report : $REPORT"
```

```
echo "  Memory : $MEMORY"
```

```
echo "  Output : $SRC_DIR"
```

```
echo "=====
```

```
echo ""
```